
青 岛 科 技 大 学

移动软件开发课程报告

题 目 奶茶点单系统

指导教师 解本巨

学生姓名 魏鹏超

学生学号 2208010423

信息科学技术 学院 计算机科学与技术 专业 224 班

2025 年 7 月 14 日

成绩评分表

选题 (10)	中文摘要 (10)	英文摘要 (10)	正文 (60)	参考文献 (10)	总分

奶茶点单系统

摘要

随着新式茶饮行业的迅猛发展，消费者对**便捷、高效的点单体验**提出了更高的要求。尤其在门店运营高峰期，订单量激增易导致系统卡顿、响应迟缓，严重影响用户体验与商家运营效率。因此，构建一套**支持高并发、响应快速、功能完善**的智能奶茶点单系统，具有重要的现实意义。

本系统采用**前后端分离架构**，前端以 **Vue3** 和 **UniApp** 为核心框架，支持多端访问（Web、微信小程序等），提升了系统的兼容性与用户覆盖率。后端基于 **Spring Boot** 构建，结合模块化设计，具备良好的扩展性与维护性，业务逻辑清晰，响应稳定。

为提升系统在高并发场景下的响应能力，系统集成 **Redis 缓存机制**，在用户登录、商品列表、订单处理等关键路径实现数据的快速访问，显著降低数据库负载，保障整体系统的高可用性与稳定性。

系统采用 **MySQL 数据库** 进行核心数据存储，支持对用户、商品、订单等数据的持久化管理。同时引入 **JWT (JSON Web Token) 认证机制**，实现用户权限控制与接口安全访问，确保数据传输安全可靠。

功能方面，系统涵盖了从商品展示、购物车、下单支付到后台管理、库存监控等多个模块，并内置**个性化推荐算法**，基于用户行为与商品特征提供精准推荐，优化用户体验，提升点单转化率。

综合来看，本系统不仅在技术实现上满足高性能与高并发需求，也在功能设计上全面覆盖用户与商家的核心使用场景，具备良好的推广应用价值。

关键词：UniApp, Vue3, Spring Boot, Redis, 高并发, 点单系统

Milk Tea Ordering System

ABSTRACT

With the rapid development of the new-style tea beverage industry, consumers are placing increasing demands on convenient and efficient ordering experiences. During peak business hours, a surge in orders can easily lead to system lag and slow response times, significantly affecting user satisfaction and merchant operational efficiency. Therefore, building an intelligent milk tea ordering system that supports high concurrency, fast response, and comprehensive functionality holds great practical significance.

This system adopts a **frontend-backend separated architecture**. The frontend is developed using **Vue3** and **UniApp**, enabling multi-platform access (Web, WeChat Mini Programs, etc.), which improves system compatibility and user reach. The backend is built with **Spring Boot**, featuring a modular design that ensures good scalability, maintainability, and stable business logic handling.

To enhance performance under high-concurrency scenarios, the system integrates **Redis caching**, allowing fast access to critical data such as user sessions, product listings, and order processing. This significantly reduces database load and ensures the system remains highly available and stable.

MySQL is used for persistent storage of key data, including user, product, and order information. Additionally, **JWT (JSON Web Token)** is employed for authentication and access control, securing data transmission and ensuring interface safety.

Functionally, the system includes modules for product display, shopping

cart, order placement and payment, as well as back-office features such as product management and inventory monitoring. It also incorporates a **personalized recommendation algorithm** that leverages user behavior and product features to deliver accurate suggestions, thereby optimizing user experience and increasing order conversion rates.

In summary, the system not only meets technical requirements for high performance and concurrency, but also comprehensively supports the core operational needs of both users and merchants. It demonstrates strong practical value and scalability for broader application.

Keywords: UniApp, Vue3, Spring Boot, Redis, High Concurrency, Ordering System

目录

第 1 章 绪论.....	1
1.1 研究背景与意义	1
1.2 研究目的与内容	1
第 2 章 系统需求分析.....	2
2.1 用户登录与注册	2
2.2 商品浏览与搜索	2
2.3 购物车管理	3
2.4 订单创建与支付	3
2.5 会员中心与积分管理	4
2.6 优惠券领取与使用	4
第 3 章 系统设计.....	5
3.1 系统架构设计	5
3.1.1 后端服务	5
3.1.2 管理端前端	5
3.1.3 客户端	6
3.2 uni-app 架构设计.....	6
3.2.1 组件化开发	7
3.2.2 页面路由与导航	7
3.2.3 状态管理	7
第 4 章 技术选型.....	8
4.1 开发工具与环境	8
4.2 技术栈	9
第 5 章 系统实现.....	11
5.1 页面布局	11
5.1.1 登录界面	11
5.1.2 安卓端首页	12
5.1.3 点单页面	12

5.1.4 用户信息界面	13
5.1.5 会员中心界面	14
5.1.6 购物车界面	15
5.1.7 商品搜索界面	15
5.1.8 创建订单界面	16
5.1.9 退单界面	16
5.2 组件开发	17
5.2.1 组件划分	17
5.2.2 组件使用	17
5.2.3 组件通信	19
5.2.4 组件样式	20
5.3 详细设计	20
5.3.1 数据库设计	20
5.3.2 用户认证	21
5.3.3 商品浏览	22
5.3.4 购物车	23
5.3.5 订单管理	25
5.3.6 会员中心	26
5.3.7 个人中心	28
5.4 Android 端打包	30
第 6 章 总结与展望	31
6.1 总结	31
6.2 系统的不足与改进方向	31
6.3 未来展望	32

第1章 绪论

1.1 研究背景与意义

随着互联网与移动设备的广泛普及，**在线购物**逐渐成为用户首选的消费方式，其市场规模不断扩大、渗透率持续提高。在此背景下，奶茶等新式茶饮行业也迎来了数字化转型的发展机遇。相比传统线下点单方式，线上点单系统不仅提供了**便捷、高效**的购物体验，还为用户带来了**更丰富的商品选择与个性化服务**。

当前消费者的需求日益呈现出**多样化、个性化**趋势，如何通过智能推荐提高用户粘性与转化率，成为各类电商平台和零售系统的研究热点。同时，面对**高并发访问、大数据处理**等复杂场景，系统必须具备**高性能、高稳定性与可扩展性**，以保障在用户数量激增时依然保持流畅的操作体验。

因此，设计并实现一套**能够应对高并发场景**的奶茶点单系统，不仅具有重要的**工程实践价值**，也为在线零售系统的智能化升级提供了具有代表性的**理论研究基础**。

1.2 研究目的与内容

本课题旨在开发一套功能完备、性能优良的**智能奶茶点单系统**，服务于门店与用户两端，提升商家运营效率与用户消费体验。研究目标包括：

- 构建一个**高并发、高响应能力**的系统架构，支持大规模用户并发访问；
- 实现一套**模块化、可维护、可拓展**的点单系统，覆盖商品管理、订单处理、库存监控、用户管理等核心功能；
- 应用 **Redis 缓存机制与 JWT 安全认证机制**，确保安全性与高可用性；
- 针对多端需求，设计基于 **Vue3 + UniApp** 的前端界面，提升系统的易用性与平台兼容性。

综上所述，本研究以“高并发支撑能力+个性化推荐”为技术核心，构建一个面向新式茶饮行业的现代化在线点单解决方案，兼顾**系统性能、用户体验与商业价值**。

第2章 系统需求分析

2.1 用户登录与注册

本系统为用户提供了完善的登录与注册功能，以保障用户身份的唯一性和账户安全性。用户可通过手机号+验证码进行快捷注册，首次登录时需设置密码，后续登录可选择使用账号密码或验证码两种方式。系统对用户输入信息进行前端与后端双重校验，防止恶意注册和信息伪造。注册成功后，用户信息将存储于数据库中，并建立对应的会员身份档案。

登录功能支持用户身份验证与会话管理，用户登录成功后可访问购物车、订单管理、会员中心等模块，未登录状态下仅可浏览商品信息。系统还支持“记住我”与自动登录功能，通过 token 或 JWT 实现用户身份的持久化认证。同时，对于注册用户的密码信息，系统采用加密算法进行存储处理，增强系统的安全性与隐私保护能力。

2.2 商品浏览与搜索

系统为用户提供直观、丰富的商品浏览与检索体验。在首页及商品页中，用户可以浏览各类奶茶商品，支持按照商品分类（如“奶盖系列”、“水果茶系列”、“经典奶茶”、“新品推荐”等）进行筛选查看。商品展示采用卡片式布局，内容包含商品图片、名称、售价、标签（如“热销”、“新品”）、月销量等关键信息，增强用户的购买欲望。

此外，系统集成了模糊搜索功能，用户可通过关键词快速搜索目标商品，并支持联想补全，提高搜索效率。为了提升用户体验，系统还提供多种排序方式，如按照“价格升序”、“销量优先”、“评分优先”等进行排序，满足用户多样化的浏览需求。点击商品卡片后，用户可进入商品详情页面，查看更详细的信息如：配料表、可选规格（大小杯）、甜度、冰量等，支持自定义搭配，提升点单灵活性。

2.3 购物车管理

购物车模块是连接商品浏览与订单提交的核心组件，系统支持用户在浏览商品时将所选商品添加至购物车，并在购物车中进行统一管理。用户可以在购物车中查看当前所有待结算商品的信息，包括商品名称、规格、单价、数量、小计金额等内容，系统实时汇总购物总价，便于用户核对。

用户可通过“+”、“-”按钮调整商品数量，或直接删除不需要的商品项。购物车数据默认保存在本地缓存中，若用户已登录，则购物车内容将自动与服务器同步保存，便于在多设备或不同时间段内保持一致性。此外，系统还提供“一键清空购物车”功能，提升使用效率。系统在用户提交订单之前对购物车内容进行校验，确保商品状态与库存信息的有效性，避免下单失败或库存异常。

2.4 订单创建与支付

订单管理模块是系统的核心业务功能之一，用户在购物车确认商品后可点击“提交订单”按钮，进入订单创建与支付流程。系统会根据用户所选商品信息、优惠券、使用积分、收货地址等数据生成订单详情，并生成唯一订单编号。用户可在提交前再次确认配送信息、支付金额与备注内容。

支付环节系统支持主流支付方式模拟，如微信支付、支付宝等（如对接实际支付接口，可扩展至真实支付服务），并通过支付状态回调确认订单支付结果。订单状态初始为“待支付”，支付完成后自动更新为“制作中”或“待配送”。后台管理端可根据订单状态进行分类管理，包括接单、完成、配送、退款等多种状态的处理。

用户可在“我的订单”页面查看全部订单记录，包括每笔订单的详细信息、下单时间、金额、状态和配送进度等。为防止恶意下单，系统也对异常订单行为进行限制，如同一用户短时间内大量取消订单、支付失败频繁等，提升系统的稳定性与运营效率。

2.5 会员中心与积分管理

会员中心是用户的个人信息与交易记录管理平台，用户登录后可在该模块中查看和编辑个人基本信息（如昵称、手机号、收货地址等），同时还可以查看订单历史、积分余额、优惠券情况、收藏商品等内容。该模块有效整合了用户在平台上的全部行为记录，是用户交互体验的核心部分。

系统设有会员等级制度，根据用户的累计消费金额划分等级（如普通会员、银卡会员、金卡会员），不同等级可享受不同优惠政策与积分获取倍率。积分系统作为促销与用户粘性管理的重要手段，用户每完成一笔有效订单即可获得相应的积分，积分可用于后续订单抵扣金额、兑换商品或参与平台活动。

在后台管理系统中，管理员可灵活配置积分获取规则、兑换规则与等级成长策略。系统还会在用户积分即将过期时发送提醒通知，提升用户活跃度与使用频率，增强会员生命周期价值。

2.6 优惠券领取与使用

优惠券功能是系统中用于增强用户留存和刺激消费的重要模块。系统支持多种类型的优惠券发放与使用规则，包括满减券（满 20 减 5）、折扣券（9 折优惠）、新用户注册礼包、生日专属券等。用户可在“优惠中心”页面中领取可用的优惠券，系统会自动根据用户身份、等级或活动条件发放对应券包。

在结算订单时，系统会根据订单金额和商品类型智能判断用户所持优惠券是否满足使用条件，并自动推荐最优优惠方案。用户可手动选择是否使用优惠券，同时支持优惠券与积分叠加使用。每张优惠券都有明确的使用规则，包括使用时间、适用门店、适用商品、最低消费门槛、是否可叠加等。

后台管理系统支持优惠券的创建、投放、核销统计等操作，可按区域、用户群体、活动主题进行精准投放，并实时监控使用效果，为商家提供数据支持。整体设计实现了优惠券从发放、使用到核销的完整闭环，有效提升了用户的购买意愿与复购率。

第3章 系统设计

3.1 系统架构设计

奶茶点单系统采用三端分离的架构模式，包括后端服务、管理端前端和客户端，各端职责明确，协同工作，为用户提供完整的在线点单体验。

3.1.1 后端服务

技术栈：基于 Spring Boot 3.4.4、Java 17、Spring Data JPA、Spring Security + JWT、MySQL 8.0 和 Redis 构建。

架构风格：采用 RESTful API 设计风格，为前端提供统一的数据接口。

主要功能：

用户管理：用户注册、登录、信息管理等。

商品管理：商品信息维护、分类管理、规格管理等。

订单管理：订单创建、支付、状态更新、退款管理等。

会员管理：会员等级、积分管理、优惠券管理等。

数据统计：销售额、订单量、用户增长等数据统计。

部署方式：可部署在云服务器或本地服务器，通过 Nginx 等反向代理提供服务。

3.1.2 管理端前端

技术栈：基于 Vue 3、Vite、Element Plus、Pinia、Vue Router 和 Axios 构建。

主要功能：

数据统计：销售额、订单量、用户增长统计。

商品管理：商品添加、编辑、上下架。

分类管理：商品分类管理。

规格管理：定义商品规格、配料选项。

订单管理：订单处理、状态更新、退款管理。

会员管理：会员信息、消费记录。

优惠券管理：创建优惠券、设置规则。

系统设置：店铺信息设置、权限管理。

部署方式：可部署在静态资源服务器上，通过 CDN 加速访问。

3.1.3 客户端

技术栈：基于 uni-app 框架，可编译为微信小程序、H5、Android 等多个平台。

主要功能：

用户认证：登录、注册、密码找回。

商品浏览：分类展示、商品详情、搜索。

购物车：添加商品、调整规格、配料添加。

订单管理：创建订单、支付、查看订单状态。

会员中心：积分查询、签到、会员等级。

优惠券：领取优惠券、使用优惠券。

个人中心：地址管理、账户设置。

部署方式：

微信小程序：通过微信公众平台发布。

H5：部署在 Web 服务器上，通过域名访问。

Android：打包成 APK 文件，通过应用商店或分发平台发布。

通过三端分离的架构设计，系统实现了前后端解耦，提高了可维护性和可扩展性，为用户提供了良好的使用体验。

3.2 uni-app 架构设计

uni-app 客户端采用组件化开发模式，结合页面路由与导航以及状态管理，构建高效、可维护的多端应用。

3.2.1 组件化开发

组件划分：将页面拆分为独立的、可复用的组件，例如商品列表组件、商品详情组件、购物车组件、订单组件等。

组件通信：通过 props 传递数据，使用 emit 触发事件，实现组件间的通信。

自定义组件：根据业务需求，封装自定义组件，提高代码复用率和可维护性。

uni-ui 组件库：利用 uni-ui 提供的丰富组件，快速搭建用户界面，减少重复开发工作。

3.2.2 页面路由与导航

页面组织：按照功能模块组织页面，例如首页、商品分类页、商品详情页、购物车页、订单确认页、用户中心等。

路由配置：通过 pages.json 文件配置页面路由，定义页面路径、页面样式等。

页面跳转：使用 uni.navigateTo、uni.redirectTo、uni.switchTab 等 API 实现页面跳转和导航。

导航栏定制：通过 navigationBarTitleText、navigationBarBackgroundColor 等配置项，定制导航栏样式。

3.2.3 状态管理

Vuex：选择 Vuex 作为状态管理工具，集中管理应用的状态数据。

状态定义：定义全局状态，例如用户信息、购物车数据、商品列表等。

状态修改：通过 mutations 修改状态，确保状态的可预测性和可维护性。

状态获取：通过 getters 获取状态，方便在组件中使用状态数据。

异步操作：通过 actions 处理异步操作，例如 API 调用、数据提交等。

第4章 技术选型

4.1 开发工具与环境

IDE: HBuilderX

Android Studio: 用于 Android 原生模块开发

UniApp 是一个使用 Vue.js 开发所有前端应用的框架，可以编译到 iOS、Android、H5、以及各种[小程序](#)等多个平台。HBuilderX 是 DCloud 推出的一款轻如羽毛、快如闪电的前端开发工具，支持 Vue.js、uni-app 等前端框架，内置了丰富的插件和工具，极大地提高了开发效率。

(一)使用 HBuilderX 创建并运行 UniApp 项目的步骤如下：

1. 打开 HBuilderX 编辑器，点击顶部菜单栏的“文件”->“新建”->“项目”。
2. 在弹出的“新建项目”对话框中，选择“UniApp 项目”，然后点击“下一步”。
3. 在“UniApp 项目”配置页面，你需要填写项目名称、项目目录、模板选择等信息。其中，模板选择可以根据你的需求选择，这里我们选择默认的“uni-app 项目模板”。
4. 点击“创建”按钮，HBuilderX 会自动为你创建一个新的 UniApp 项目。

(二)运行 uniapp 项目：

1. 在 HBuilderX 的编辑器中，你可以看到新创建的 UniApp 项目结构。找到“pages”目录下的“index”文件夹，打开“index.vue”文件，这是项目的首页。
2. 在顶部菜单栏点击“运行”->“运行到小程序模拟器”->“微信开发者工具”。当然，你也可以选择运行到其他平台，比如 H5、APP 等。
3. HBuilderX 会自动打开微信开发者工具，并将项目代码编译并运行。你可以在微信开发者工具中看到项目的运行效果。

(三)注意事项：

1. 在开发过程中，确保你的 HBuilderX 和微信开发者工具等开发工具都是最新版本，以保证开发效率和稳定性。

2. 在编写代码时，要注意代码的规范和可读性，避免因为代码问题导致项目运行出错。
3. 在运行项目之前，最好先进行一次代码检查，确保没有语法错误或逻辑错误。

通过以上步骤，我们成功使用 HBuilderX 创建并运行了 UniApp 项目。UniApp 作为跨平台的前端框架，具有非常好的兼容性和扩展性，可以帮助我们快速开发多端应用。HBuilderX 作为强大的开发工具，提供了丰富的插件和工具，极大地提高了我们的开发效率。



图 1 uni-app 功能框架图

4.2 技术栈

框架：uni-app

使用 Vue.js 语法进行跨平台开发，一套代码可编译为 Android 应用。

通过条件编译，可以针对 Android 平台进行特定功能的适配和优化。

平台：Android

通过 HBuilderX 打包生成 Android APK 文件。

可发布到应用商店或通过分发平台进行安装。

UI 组件库: uni-ui

提供丰富的跨平台 UI 组件，快速构建美观、一致的用户界面。

支持自定义组件，满足特定业务需求。

状态管理: Vuex 或 Pinia (可选)

集中管理应用状态，方便组件间数据共享和状态同步。

Vuex 适用于中大型应用，Pinia 更轻量级，易于上手。

网络请求: uni.request

提供统一的 API，发起 HTTP 请求，与后端服务进行数据交互。

支持 Promise 风格的异步操作，方便处理网络请求结果。

原生 API 调用:

通过 `uni.requireNativeComponent` 引入 Android 原生组件，实现更高级的功能和性能优化。

例如，调用 Android 原生支付 SDK、地图 SDK 等。

打包与发布:

使用 HBuilderX 工具，将 uni-app 项目打包成 Android APK 文件。

可以通过应用商店或分发平台发布应用。

第5章 系统实现

5.1 页面布局

5.1.1 登录界面



图 2 安卓端用户登录界面

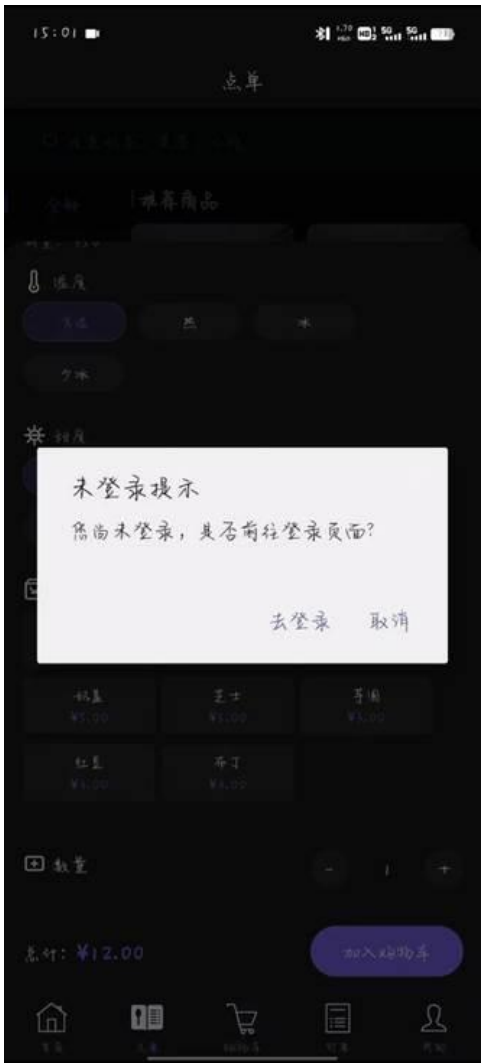


图 3 安卓端登录提示

5.1.2 安卓端首页



图 4 安卓端首页

5.1.3 点单页面



图 5 点单总览界面



图 6 点单详细界面 1

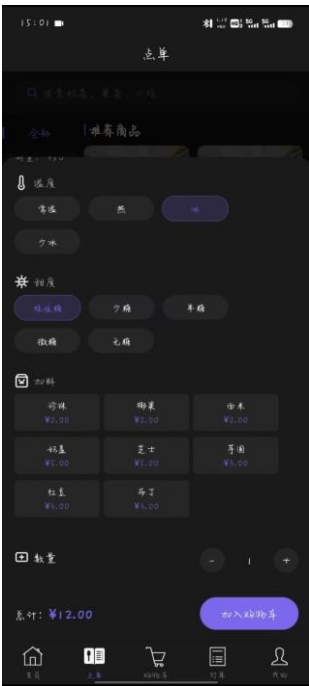


图 7 点单详细界面 2

5.1.4 用户信息界面

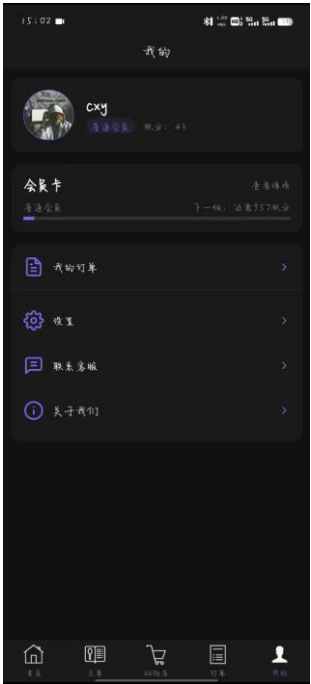


图 8 个人信息界面

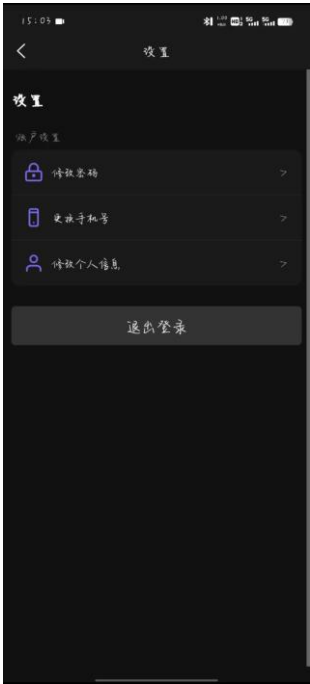


图 9 信息修改界面



图 10 密码修改界面

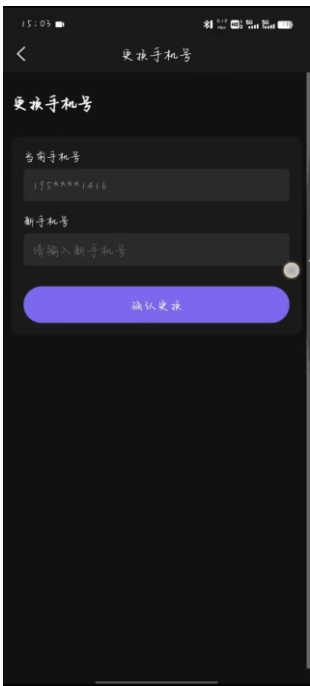


图 11 手机号修改界面

5.1.5 会员中心界面

图 12 展示了优惠券领取功能；

图 13 展示了优惠券列表；

图 14 展示了用户积分管理功能。



图 12 优惠券领取



图 13 优惠券列表



图 14 用户积分管理

5.1.6 购物车界面

我们选择两杯奶茶下单，购物车界面如下：

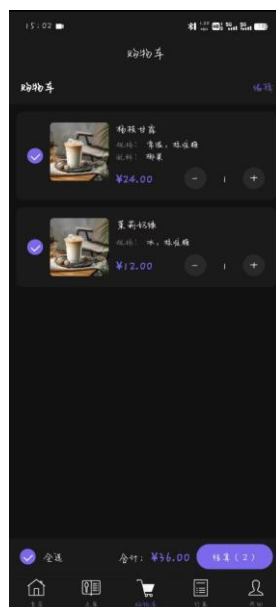


图 15 购物车界面

5.1.7 商品搜索界面

我们以“脏脏茶”为例：

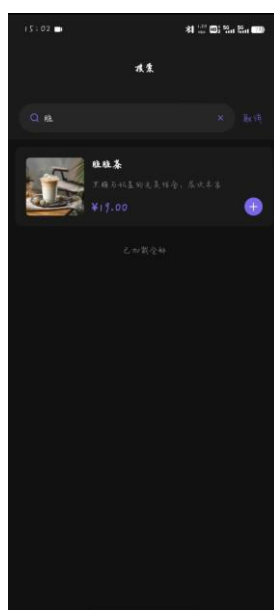


图 16 商品搜索界面

5.1.8 创建订单界面



图 17 提交订单

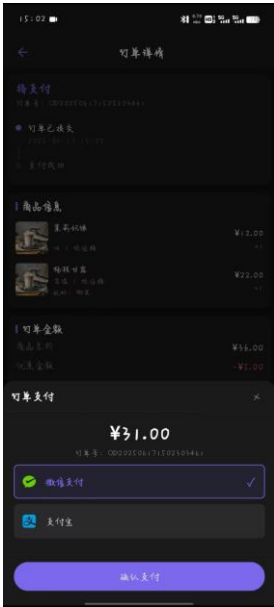


图 18 支付订单



图 19 创建订单

5.1.9 退单界面



图 20 退单理由



图 21 申请退单

5.2 组件开发

在 uni-app 客户端的 Android 端开发中，组件开发是构建用户界面的核心环节。通过组件化，可以将页面拆分成独立的、可复用的模块，提高代码的可维护性和开发效率。

5.2.1 组件划分

通用组件：

- **mt-button**：自定义按钮组件，用于统一按钮样式和交互。
- **mt-card**：卡片组件，用于展示商品信息、订单信息等。
- **lazy-image**：图片懒加载组件，优化图片加载性能。
- **skeleton**：骨架屏组件，在数据加载时提供占位效果，提升用户体验。
- **uni-popup**：uni-app 官方提供的弹窗组件，用于显示商品详情、支付选项等。

业务组件：

- **商品列表组件**：展示商品列表，支持分页加载。
- **商品详情组件**：展示商品详细信息，包括图片、价格、规格等。
- **购物车组件**：展示购物车商品，支持数量调整、删除等操作。
- **订单组件**：展示订单信息，支持查看订单状态、取消订单等操作。

5.2.2 组件使用

引入组件：在页面或组件中，使用 `import` 语句引入需要的组件。

注册组件：在 `components` 选项中注册引入的组件。

使用组件：在模板中使用组件标签，并传递必要的数据。

```
01 //filepath:
02 //f:\Repository\Milk-Tea-Order-System\milktea-uniapp\pages\index\index.vue
03 <template>
04   <view class="home-container">
05     <mt-card title="门店信息" :content="shopInfo.notice"></mt-card>
06   </view>
07 </template>
08
09 <script>
10 import mtCard from '@components/mt-card/mt-card.vue';
11
12 export default {
13   components: {
14     mtCard
15   },
16   data() {
17     return {
18       shopInfo: {
19         shopName: '奶茶店',
20         notice: '欢迎光临'
21       }
22     };
23   }
24 };
</script>
```

5.2.3 组件通信

父组件向子组件传递数据：通过 `props` 选项定义组件的属性，父组件通过属性向子组件传递数据。

子组件向父组件传递数据：通过 `emit` 方法触发自定义事件，并传递数据给父组件。

```
01 //                                                     filepath:
02 f:\Repository\Milk-Tea-Order-System\milktea-uniapp\pages\index\index.vue
03 <template>
04     <view class="home-container">
05         <mt-button text="点单" @click="goToMenu"></mt-button>
06     </view>
07 </template>
08 <script>
09 import mtButton from '@components/mt-button/mt-button.vue';
10 export default {
11     components: {
12         mtButton
13     },
14     methods: {
15         goToMenu() {
16             uni.switchTab({
17                 url: '/pages/menu/menu'
18             });
19         }
20     }
21 };
```

```
</script>
```

5.2.4 组件样式

scoped 样式：在组件的 `<style>` 标签中添加 `scoped` 属性，使样式只在当前组件生效，避免样式冲突。

全局样式：在 [App.vue](#) 中定义全局样式，应用于所有页面和组件。

主题定制：通过修改 `uni.scss` 文件，定制主题颜色、字体等。

5.3 详细设计

5.3.1 数据库设计

数据库是系统的数据存储中心，合理的数据库设计能够保证数据的完整性、一致性和高效访问。系统包含以下主要数据表：

- **用户表 (user)：**存储用户基本信息，例如用户 ID、用户名、密码、手机号、注册时间等。
- **管理员表 (admin)：**存储管理员信息，例如管理员 ID、用户名、密码等。
- **商品表 (product)：**存储商品信息，例如商品 ID、商品名称、商品描述、商品价格、商品图片、所属分类 ID、是否上架等。
- **分类表 (category)：**存储商品分类信息，例如分类 ID、分类名称、分类描述等。
- **规格表 (specification)：**存储商品规格信息，例如规格 ID、商品 ID、规格名称、规格值等。
- **配料表 (ingredient)：**存储可选配料信息，例如配料 ID、配料名称、配料价格等。
- **订单表 (order)：**存储订单主信息，例如订单 ID、用户 ID、订单编号、订单总金额、支付金额、支付方式、订单状态、创建时间等。
- **订单详情表 (order_detail)：**存储订单商品明细，例如订单详情 ID、订单 ID、商品 ID、商品数量、商品价格等。

-
- **购物车表 (cart)**：存储用户购物车信息，例如购物车 ID、用户 ID、商品 ID、商品数量等。
 - **优惠券表 (coupon)**：存储优惠券模板信息，例如优惠券 ID、优惠券名称、优惠券描述、优惠券类型、优惠券面额、使用条件、有效期等。
 - **用户优惠券表 (user_coupon)**：存储用户领取的优惠券，例如用户优惠券 ID、用户 ID、优惠券 ID、领取时间、使用状态等。
 - **积分记录表 (point_record)**：存储用户积分变动记录，例如积分记录 ID、用户 ID、积分变动数量、变动原因、变动时间等。
 - **会员等级表 (member_level)**：存储会员等级配置，例如会员等级 ID、会员等级名称、所需积分、享受折扣等。
 - **签到表 (checkin)**：存储用户签到记录，例如签到 ID、用户 ID、签到时间等。
 - **店铺信息表 (shop_info)**：存储店铺基本信息，例如店铺 ID、店铺名称、店铺地址、店铺电话、店铺公告等。

5.3.2 用户认证

用户认证模块主要负责用户的注册、登录和密码找回功能，确保用户身份的合法性。

注册 ([register.vue](#))

界面：提供用户名、密码、确认密码等输入框，以及用户协议勾选框。

流程：

用户填写注册信息。

前端进行表单验证，例如用户名是否为空、密码强度是否足够等。

调用后端注册 API (/api/register)，传递注册信息。

后端验证用户名是否已存在，如果不存在则创建新用户，并返回注册结果。

前端根据注册结果显示提示信息，例如注册成功或失败。

注册成功后，跳转到登录页面。

登录 ([login.vue](#))

界面：提供用户名、密码等输入框，以及记住密码选项。

流程：

用户填写登录信息。

前端进行表单验证，例如用户名和密码是否为空。

调用后端登录 API (`/api/login`)，传递登录信息。

后端验证用户名和密码是否正确，如果正确则生成 JWT Token，并返回给前端。

前端将 JWT Token 存储在本地存储 (`uni.setStorageSync('token', token)`)，并跳转到首页。

密码找回

流程：

用户在登录页面点击“忘记密码”链接。

跳转到密码找回页面，输入注册时使用的手机号或邮箱。

前端验证手机号或邮箱格式是否正确。

调用后端发送验证码 API，向用户手机或邮箱发送验证码。

用户输入收到的验证码，并设置新密码。

前端验证验证码和密码格式是否正确。

调用后端重置密码 API，传递手机号或邮箱、验证码和新密码。

后端验证验证码是否正确，如果正确则重置用户密码，并返回重置结果。

前端根据重置结果显示提示信息，例如重置成功或失败。

重置成功后，跳转到登录页面。

5.3.3 商品浏览

商品浏览模块主要负责展示商品分类和商品列表，方便用户浏览和选择商品。

商品分类展示 ([menu.vue](#))

界面：展示商品分类列表，例如人气热卖、水果茶、奶茶系列、咖啡

系列、特调饮品等。

流程：

页面加载时，调用后端获取商品分类 API (/api/categories)。

后端返回商品分类列表。

前端将商品分类列表渲染到页面上，并为每个分类创建一个导航链接。

商品列表展示 ([menu.vue](#))

界面：展示指定分类下的商品列表，包括商品图片、商品名称、商品价格等。

流程：

用户点击某个商品分类，或在搜索框中输入关键词进行搜索。

前端调用后端获取商品列表 API (/api/products)，传递分类 ID 或搜索关键词。

后端根据分类 ID 或搜索关键词查询商品列表，并返回给前端。

前端将商品列表渲染到页面上，并为每个商品创建一个链接，点击可以查看商品详情。

商品详情展示 ([menu.vue](#), [search.vue](#))

界面：展示商品的详细信息，包括商品图片、商品名称、商品描述、商品价格、商品规格、可选配料等。

流程：

用户点击某个商品，进入商品详情页面。

前端调用后端获取商品详情 API (/api/product/{productId})，传递商品 ID。

后端根据商品 ID 查询商品详情，并返回给前端。

前端将商品详情渲染到页面上，并提供添加到购物车按钮。

5.3.4 购物车

商品浏览模块主要负责展示商品分类和商品列表，方便用户浏览和选择商品。

商品分类展示 ([menu.vue](#))

界面：展示商品分类列表，例如人气热卖、水果茶、奶茶系列、咖啡系列、特调饮品等。

流程：

页面加载时，调用后端获取商品分类 API (/api/categories)。

后端返回商品分类列表。

前端将商品分类列表渲染到页面上，并为每个分类创建一个导航链接。

商品列表展示 ([menu.vue](#))

界面：展示指定分类下的商品列表，包括商品图片、商品名称、商品价格等。

流程：

用户点击某个商品分类，或在搜索框中输入关键词进行搜索。

前端调用后端获取商品列表 API (/api/products)，传递分类 ID 或搜索关键词。

后端根据分类 ID 或搜索关键词查询商品列表，并返回给前端。

前端将商品列表渲染到页面上，并为每个商品创建一个链接，点击可以查看商品详情。

商品详情展示 ([menu.vue](#), [search.vue](#))

界面：展示商品的详细信息，包括商品图片、商品名称、商品描述、商品价格、商品规格、可选配料等。

流程：

用户点击某个商品，进入商品详情页面。

前端调用后端获取商品详情 API (/api/product/{productId})，传递商品 ID。

后端根据商品 ID 查询商品详情，并返回给前端。

前端将商品详情渲染到页面上，并提供添加到购物车按钮。

5.3.5 订单管理

订单管理模块主要负责展示用户的订单列表和订单详情，方便用户查看订单状态和进行售后服务。

订单列表展示 ([order.vue](#))

界面：展示用户的订单列表，包括订单号、订单金额、订单状态、创建时间等。

流程：

用户点击底部导航栏的“订单”按钮，进入订单列表页面。

前端调用后端获取订单列表 API (`/api/orders`)，传递用户 ID 和订单状态。

后端根据用户 ID 和订单状态查询订单列表，并返回给前端。

前端将订单列表渲染到页面上，并为每个订单创建一个链接，点击可以查看订单详情。

订单详情展示 ([detail.vue](#))

界面：展示订单的详细信息，包括订单号、订单金额、订单状态、创建时间、商品列表、收货地址、支付方式等。

流程：

用户点击某个订单，进入订单详情页面。

前端调用后端获取订单详情 API (`/api/order/{orderId}`)，传递订单 ID。

后端根据订单 ID 查询订单详情，并返回给前端。

前端将订单详情渲染到页面上，并提供取消订单、支付订单、确认收货、申请退款等操作按钮。

取消订单 ([detail.vue](#))

流程：

用户在订单详情页面点击“取消订单”按钮。

前端调用后端取消订单 API (`/api/order/{orderId}/cancel`)，传递订单 ID。

后端取消订单，并返回取消结果。

前端根据取消结果显示提示信息，并更新页面上的订单状态。

支付订单 ([detail.vue](#))

流程:

用户在订单详情页面点击“支付订单”按钮。

前端调用后端获取支付信息 API
(/api/order/{orderId}/pay)，传递订单 ID 和支付方式。

后端生成支付信息，并返回给前端。

前端根据支付信息引导用户完成支付。

确认收货

流程:

用户在订单详情页面点击“确认收货”按钮。

前端调用后端确认收货 API
(/api/order/{orderId}/confirm)，传递订单 ID。

后端确认收货，并返回确认结果。

前端根据确认结果显示提示信息，并更新页面上的订单状态。

申请退款

流程:

用户在订单详情页面点击“申请退款”按钮。

前端调用后端申请退款 API
(/api/order/{orderId}/refund)，传递订单 ID 和退款原因。

后端处理退款申请，并返回处理结果。

前端根据处理结果显示提示信息，并更新页面上的订单状态。

5.3.6 会员中心

会员中心模块主要负责展示用户的会员信息、积分信息、优惠券信息等，方便用户管理会员权益。

会员信息展示 ([member-center.vue](#), [mine.vue](#))

界面：展示用户的会员等级、当前积分、可用优惠券数量等。

流程:

用户点击底部导航栏的“我的”按钮，进入个人中心页面，然后点击“会员中心”链接。

前端调用后端获取会员信息 API (/api/member)。

后端查询用户的会员信息，并返回给前端。

前端将会员信息渲染到页面上。

积分记录展示 ([member-center.vue](#))

界面: 展示用户的积分变动记录，包括积分变动数量、变动原因、变动时间等。

流程:

用户在会员中心页面点击“积分记录”链接。

前端调用后端获取积分记录 API (/api/points)，传递用户 ID。

后端根据用户 ID 查询积分记录，并返回给前端。

前端将积分记录渲染到页面上。

优惠券展示 ([member-center.vue](#))

界面: 展示用户可用的优惠券列表，包括优惠券名称、优惠券描述、优惠券类型、优惠券面额、使用条件、有效期等。

流程:

用户在会员中心页面点击“优惠券”链接。

前端调用后端获取优惠券 API (/api/coupons)，传递用户 ID。

后端根据用户 ID 查询优惠券列表，并返回给前端。

前端将优惠券列表渲染到页面上。

签到

流程:

用户在会员中心页面点击“签到”按钮。

前端调用后端签到 API (/api/checkin)，传递用户 ID。

后端记录用户签到信息，并返回签到结果。

前端根据签到结果显示提示信息，并更新页面上的签到状态。

5.3.7 个人中心

个人中心模块主要负责展示用户的个人信息，并提供修改个人信息、修改密码、更换手机号、退出登录等功能。

个人信息展示 (pages/user/user-info.vue, [mine.vue](#))

界面：展示用户的用户名、头像、手机号、邮箱等。

流程：

用户点击底部导航栏的“我的”按钮，进入个人中心页面。

前端调用后端获取用户信息 API (/api/user)。

后端查询用户信息，并返回给前端。

前端将用户信息渲染到页面上。

修改个人信息 (pages/user/user-info.vue)

流程：

用户在个人中心页面点击“修改个人信息”链接。

跳转到修改个人信息页面，提供用户名、头像、手机号、邮箱等输入框。

用户修改个人信息，并点击“保存”按钮。

前端验证输入信息的格式是否正确。

调用后端更新用户信息 API (/api/user)，传递修改后的用户信息。

后端更新用户信息，并返回更新结果。

前端根据更新结果显示提示信息，并更新页面上的个人信息。

修改密码 (pages/user/change-password.vue)

流程：

用户在个人中心页面点击“修改密码”链接。

跳转到修改密码页面，提供原密码、新密码、确认密码等输入框。

用户输入原密码、新密码和确认密码，并点击“保存”按钮。

前端验证输入信息的格式是否正确。

调用后端修改密码 API (/api/user/password)，传递原密码、新密码和确认密码。

后端验证原密码是否正确，如果正确则修改用户密码，并返回修改结果。

前端根据修改结果显示提示信息，并跳转到登录页面。

更换手机号 (pages/user/change-phone.vue)

流程：

用户在个人中心页面点击“更换手机号”链接。

跳转到更换手机号页面，提供新手机号和验证码输入框。

用户输入新手机号，并点击“获取验证码”按钮。

前端验证手机号格式是否正确。

调用后端发送验证码 API，向用户新手机号发送验证码。

用户输入收到的验证码，并点击“保存”按钮。

前端验证验证码格式是否正确。

调用后端更换手机号 API (/api/user/phone)，传递新手机号和验证码。

后端验证验证码是否正确，如果正确则更换用户手机号，并返回更换结果。

前端根据更换结果显示提示信息，并更新页面上的手机号。

退出登录 ([settings.vue](#))

流程：

用户在个人中心页面点击“退出登录”按钮。

前端清除本地存储中的 JWT Token (uni.removeStorageSync('token')) 和用户信息 (uni.removeStorageSync('userInfo'))。

跳转到登录页面。

5.4 Android 端打包

使用 uniapp 安心打包：

应用名称: tmp_uni_default
应用版本号: 1.00

☒ Android (apk包) ☐ iOS (ipa包)

Android设置 iOS设置

Android包名: io.test.test

[Android证书使用指南](#)

☐ 使用自有证书 [如何生成证书](#) ☐ 使用云端证书 [详情](#)
☒ 使用公共测试证书 [详情](#) ☐ 使用DCloud老版证书

证书别名:
证书私钥密码:
证书文件: [浏览...](#)

渠道包 [渠道包制作指南](#)

☐ 无 ☐ GooglePlay(AAB) ☐ 应用宝 ☐ 360应用市场
☐ 华为应用商店 ☐ 小米应用商店 ☐ OPPO ☐ VIVO

☒ 打正式包 ☐ 打自定义调试基座 (iOS的Safari调试需要用苹果开发证书打包) [什么是自定义调试基座?](#)

原生混淆
☐ 对配置的js/nvue文件进行原生混淆 [配置指南](#)

广告联盟
加入 uni-AD 广告联盟, 帮助你的App变现。 [官网介绍](#) [如何开通?](#)

开通基础广告:
☒ 基础开屏广告 ☐ 悬浮红包广告 ☐ push广告

集成增强广告SDK ([组件文档](#)):
☐ 腾讯优量汇 ☐ 穿山甲 ☐ 快手广告联盟 ☐ 360广告联盟
☐ Signoby广告联盟 ☐ 华为广告联盟 ☐ 百度百青藤广告联盟

集成内容场景变现SDK:
☐ 快手内容联盟 [详情](#) ☐ 互动游戏 (变现猫) [详情](#)

换量联盟

☒ 传统打包 (上传代码及证书, DCloud承诺不保留) ☒ 快速安心打包 (不上传代码及证书, 打包更快) [详情](#) [打包 \(P\)](#)

图 22 apk 打包

安心打包优势

- 更安全：打包时不提交应用代码、证书等信息
- 更快速：非首次打包时不用提交云端打包机排队等待，本地直接出包
- 省流量：减少了打包时提交打包资源，非首次打包时不用下载原生代码包
- 更便宜：除非使用了体积很大的本地原生插件，否则将难以突破 40M 的免费打包体积阈值。开发者和 DCloud 的成本双下降

第6章 总结与展望

6.1 总结

奶茶点单系统是一个功能完善、易于使用的在线点单平台，实现了用户端和管理端的核心功能。通过采用前后端分离的架构、Spring Boot、Vue 3 和 uni-app 等现代技术栈，系统具有良好的可维护性、可扩展性和跨平台能力。

技术亮点：

三端分离架构：前后端分离，职责明确，易于维护和扩展。

Spring Boot：快速构建 RESTful API，简化开发流程。

Vue 3 + Element Plus：构建美观、高效的管理端界面。

uni-app：一套代码多端运行，降低开发成本。

JWT 认证：保障用户数据安全。

功能亮点：

用户端：提供便捷的商品浏览、购物车管理、订单管理、会员中心等功能。

管理端：提供全面的商品管理、订单管理、会员管理、数据统计等功能。

部署亮点：

后端：可部署在云服务器或本地服务器。

管理端：可部署在静态资源服务器上。

客户端：可发布为微信小程序、H5 应用或 Android 应用。

6.2 系统的不足与改进方向

尽管系统已经具备了基本的功能，但仍存在一些不足之处，需要在后续版本中进行改进：

性能优化：

数据库查询优化：针对高频查询接口，优化 SQL 语句，提升查询效率。

缓存策略优化：合理使用 Redis 缓存，降低数据库访问压力。

功能完善：

支付方式扩展：增加更多支付方式，例如支付宝、银联支付等。

营销活动支持：增加优惠券、满减、折扣等营销活动功能。

外卖配送支持：对接第三方外卖平台，提供外卖配送服务。

数据分析：提供更丰富的数据分析报表，帮助商家了解用户行为和销售情况。

用户体验优化：

界面美化：优化 UI 设计，提升用户体验。

交互优化：改进交互流程，简化操作步骤。

加载速度优化：优化页面加载速度，提升用户体验。

安全性增强：

防止 SQL 注入：对用户输入进行严格的验证和过滤，防止 SQL 注入攻击。

防止 XSS 攻击：对用户输入进行 HTML 转义，防止 XSS 攻击。

数据加密：对敏感数据进行加密存储，保障数据安全。

6.3 未来展望

未来，奶茶点单系统将继续朝着以下方向发展：

- **智能化：**
 - **个性化推荐：**根据用户历史行为和偏好，推荐个性化的商品。
 - **智能客服：**引入智能客服系统，提供 24 小时在线服务。
 - **智能库存管理：**根据销售数据预测库存需求，优化库存管理。
- **社交化：**
 - **用户评价：**支持用户对商品进行评价和分享。
 - **社交分享：**支持用户将商品分享到社交平台。
 - **社区互动：**构建用户社区，增加用户粘性。
- **平台化：**
 - **多店铺支持：**支持多个奶茶店入驻，构建平台化生态。
 - **开放 API：**提供开放 API，方便第三方应用集成。
 - **数据共享：**与供应链系统对接，实现数据共享和协同。

通过不断的技术创新和功能完善，奶茶点单系统将成为奶茶店不可或缺的数字化工具，助力商家提升运营效率和用户体验。